Library Management Application

Lucknow Public Library Network

A transactional lending, reservation and fines system built in Python with SQLite

Language / DB	Python 3.12, SQLite (relational, migration-managed)
Architecture	Domain model, repository layer, service layer, CLI
Patterns used	Repository, Unit of Work, Strategy, Factory
Tests	64 unit and integration tests, all passing
Sample data	2,000 books, 4,744 copies, 500 members, 7,079 loan-history rows

1. What I set out to build

The brief was a fictional three-branch library network in Lucknow (Hazratganj, Aminabad and Gomti Nagar) that still runs on paper registers. It needed an application that could handle lending, reservations and fines without anyone having to cross-check a notebook. I decided early that the interesting part of this project was not the screens but the rules: who may borrow what, when a book counts as late, and how to make sure a failure halfway through a transaction never leaves the data in a strange state. So I put most of my effort into the business logic, the database design and the tests, and kept the interface to a clean command-line tool.

2. How the code is organised

I split the code into layers so each piece has one job. The domain classes (*Member, Book, Copy, Loan, Reservation, Tier*) just describe things. The repository layer is the only place with SQL. The service layer holds every business rule, and the command-line interface only translates typed commands into service calls. This made testing easy: almost every rule is checked against a fresh in-memory database in a few milliseconds, with no interface involved.

Design patterns, and why each one earns its place

- **Repository:** keeps SQL out of the rules, so a rule reads like a sentence instead of a query.
- **Unit of Work:** one business action equals one database transaction. If issuing a book fails at the last step, the earlier steps are undone automatically. It also nests safely when one service calls another.
- **Strategy:** the calendar decides which days count as late, while a separate fine policy decides what those days cost. Basic members have an escalating rate, Standard and Premium a flat one. Adding a new rule means adding a small class, not editing date code.
- **Factory:** a small function turns a tier record into the right fine policy object.

3. Features delivered

- Catalogue with books and physical copies per branch; ISBN-13 check-digit validation.
- Member registration with Basic, Standard and Premium tiers, each with its own loan limit, loan period, renewal limit and fine policy.
- Issue, return and renewal with full rule enforcement.
- Reservation queues that are strictly first-come-first-served, with a 3-day hold on a returned copy and automatic hand-over if it is not collected.
- Fine calculation that skips Sundays and public holidays, slides a due date that lands on a closed day to the next open day, applies a grace period and stops at a per-tier cap.
- Search by title, author and ISBN, plus an overdue report per branch showing days late and the fine accrued so far.

4. Business rules and how they are enforced

Rule	Behaviour	Exception raised
Borrowing limit	Basic 3, Standard 5, Premium 10 books at a time	BorrowLimitExceededError
Unpaid fines	Any unpaid fine blocks issue, renewal and new reservations	UnpaidFinesError
Reserved copy	A held copy can only be issued to the member it is held for	CopyReservedError
Copy availability	On-loan or lost copies cannot be issued	CopyNotAvailableError
Membership status	Suspended or expired members cannot borrow	MembershipInactiveError
Same title twice	A member cannot hold two copies of one title	DuplicateError
Renewals	Tier limit (1/2/3); not if overdue, or if someone is waiting	RenewalNotAllowedError
Reservations	Not allowed if a copy is free, or if already queued	ReservationError / DuplicateError

Fine calculation in detail

Fines are worked out when a book comes back. The calculator first moves the due date forward if it lands on a Sunday or holiday, then counts only the open days after it. The tier's grace days are subtracted from that count (so a Standard member returning 3 open days late pays nothing, while 5 days late pays for 2). Escalating pricing doubles the daily rate after 14 billable days, and every result is clipped to the tier's cap. Money is stored as whole paise to avoid floating-point rounding surprises.

Tier	Loan limit	Loan days	Renewals	Grace	Rate/day	Cap
Basic	3	14	1	2 days	Rs 2 (x2 after 14 days)	Rs 200
Standard	5	21	2	3 days	Rs 2 flat	Rs 200
Premium	10	28	3	5 days	Rs 1.50 flat	Rs 100

5. Database, migrations and sample data

The schema is managed by numbered migration files that a small runner applies in order, each inside its own transaction. A deliberately broken migration is part of the test suite to prove that a failure leaves no half-built tables behind. Migration 001 builds the schema, 002 loads branches, tiers and holidays, and 003 generates the synthetic data with a fixed random seed so everyone gets identical results.

The messy loan history

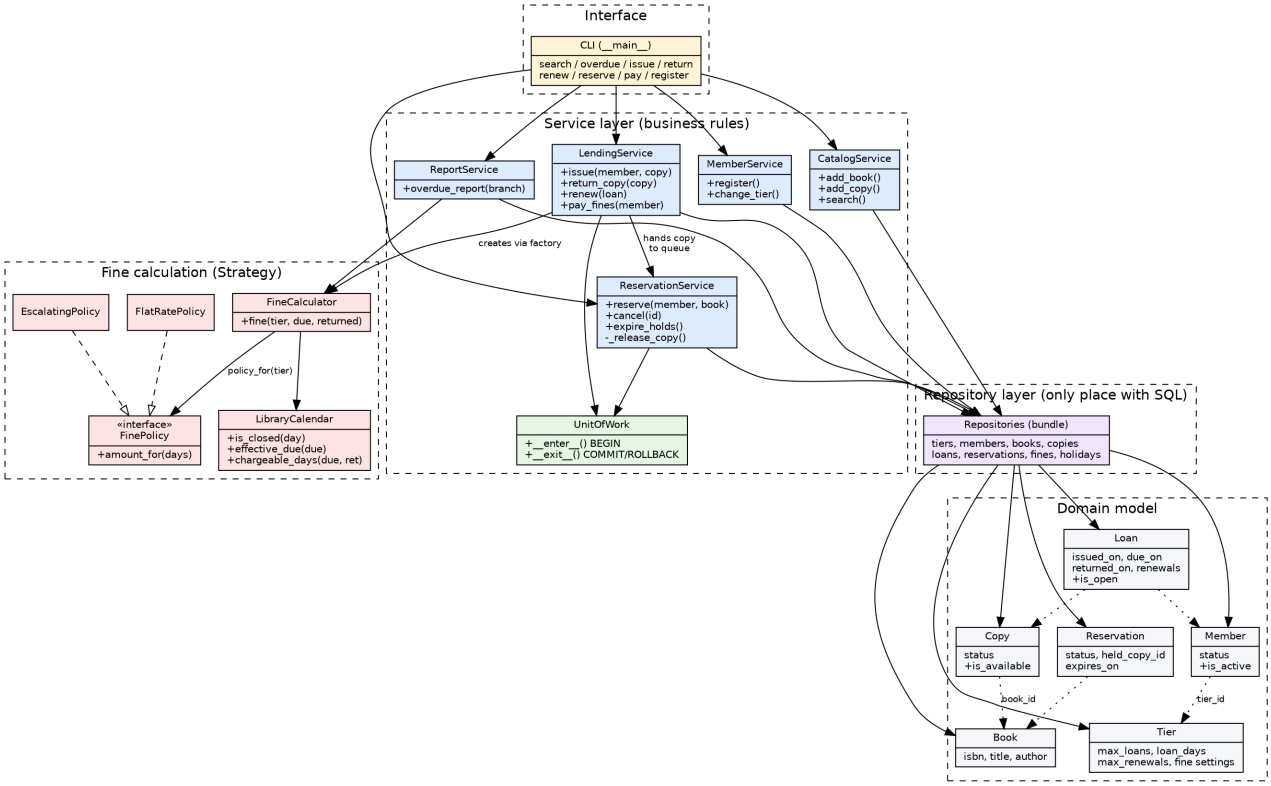
I wanted the inconsistent records to feel realistic, so I injected the mistakes a paper register would contain and then wrote a cleaning step instead of pretending the data was perfect. Raw rows land in an unconstrained staging table, and each one is validated before it reaches the real *loans* table.

Measure	Count
Raw history rows	7,079
Loans loaded after cleaning	6,907
Rejected into quarantine (with a reason)	172
Repaired (missing due date recomputed)	40
Loans still open at the cut-off date	331
Live reservations seeded	50

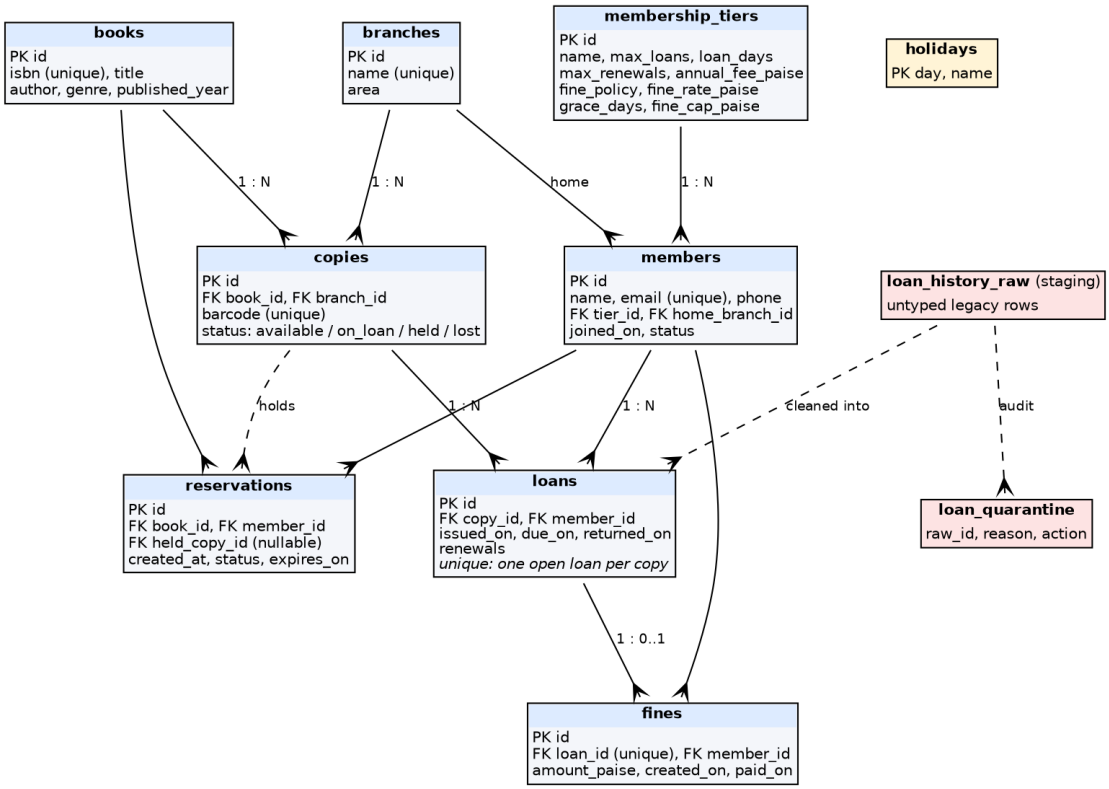
Rejection reasons include: returned before it was issued, unknown member, unknown copy, return date in the future, exact duplicates, overlapping loans, and a second open loan on the same copy. Every rejected row stays in *loan_quarantine*, so nothing disappears silently.

6. Diagrams

Class diagram



Entity-relationship diagram



7. Testing

The suite has 64 tests and all of them pass in about one second. I wrote a test for every business rule, plus integration tests that run the real migrations (including the 2,000-book load) and the command-line tool end to end.

Test file	Tests	What it covers
test_fines.py	11	Grace, caps, Sundays, holidays, due-on-closed-day, tier policies
test_lending.py	14	Tier limits (parameterised), unpaid fines, availability, inactive members, returns
test_reservations_renewals.py	14	Queue order, held copies, expiry, cancellation, every renewal restriction
test_transactions.py	5	Rollback on failure in issue and return, nesting, database safety net
test_catalog_members.py	13	Registration, validation, tier changes, ISBN, search, overdue report
test_migrations_integration.py	7	Zero-error migrations, data volumes, quarantine accounting, rollback, CLI

The most satisfying tests are the rollback ones. I force a failure in the middle of an issue or a return (using a monkeypatched repository) and then check that the copy status, loan and fine are all exactly as they were. That is the real proof that the transaction handling works.

8. Key decisions and trade-offs

- **SQLite:** zero setup for anyone reviewing the project, and it fully supports transactions. The trade-off is a single writer at a time, which suits three branches but would need a server database to scale.
- **Constraints in the database as well as the code:** partial unique indexes stop two open loans on one copy even if a service bug slips through. This duplicates a few rules, but data integrity matters more than avoiding repetition.
- **Holds as a copy status:** marking a returned copy as *held* makes the rule 'don't issue reserved copies' a single check. The queue is per title across the network, so a member may collect from another branch.
- **Fines at return time:** simple and predictable, but fines are not accrued nightly, so the overdue report calculates 'fine so far' on the fly.
- **Quarantine instead of deletion:** adds tables and migration length, but the legacy data stays auditable.
- **Command line over web UI:** the stretch goal was either; I chose the CLI so time went into rules and tests. Because the services know nothing about the interface, a web front end could be added without touching them.

9. Limitations and next steps

- No user login or staff roles; the CLI trusts its operator.

- No partial fine payments, lost-book charges or membership-fee billing.
- Holiday dates are approximate because lunar festivals shift each year.
- Next: a small web interface, scheduled fine accrual and email reminders for holds about to expire.

10. How to run it

```
python -m library_app migrate     # create DB and load sample data
python -m pytest -v               # run all 64 tests
python -m library_app search --title "Silent"
python -m library_app overdue --branch 2
```

11. What I learned

The biggest lesson was how much easier everything became once transactions and rules were separated from the interface. I also learned to treat dirty data as a normal part of the job: staging, validating and quarantining it was more reliable (and more honest) than quietly fixing or discarding rows. Finally, writing the tests alongside the rules caught real mistakes early, including some incorrect ISBN check digits in my own test fixtures, which the validator correctly rejected.